ENGINEERING MASTER DOCUMENT · B.TECH TEAM

Refill, built

How Refill is made, end to end: the architecture, the database, the frontend, the stack, the algorithms, the tests, and exactly what works today.



Case	C29 · Appointment Recovery Desk · Fynd × NMIMS Hyderabad, 17–18 October 2026
Live app	fillthechair.vercel.app · story page /splash · proof page /#/lab
Code	github.com/sirgreaseball/refill · main at commit e5daa23 (18 Oct 2026)
Film	youtu.be/5BfsKofkIHw (unlisted, 3:29)
Team	Dhruv Sonar (lead build) · Vedanth Raj (reliability, demo script) · Parth Kachhara (scenarios, real-phone tests, Lab) · Chirag Seervi (salon data, market shocks, splash page) · two MBA teammates
Data	Every name, price and booking in the demo is synthetic.

CONTENTS

What is in this document

Refill on one page 4
The three case checks and where each is proven 4
Refill in numbers 4
Architecture 5
Thin client, thick database 5
One request, step by step 6
Where state lives 6
The stack 7
Repository map 8
The rules engine and its algorithms 10
Data model 10
Validation: a bad file never gives a half-right answer 10
The fit test: one reason, in a fixed order 10
Matching: greedy, in queue order, nobody twice 11
Races: the same rule as the database 11
Notice: normal, late or past 11
Words for people 12
Worked example: Kabir's 12:00 opens up 12
Scenario files and the organiser fixture 12
The database 13
Security model 13
Tables 13
Indexes and constraints that carry guarantees 14
Concurrency: how one booking per slot is guaranteed 15
Offers: hold window, cap and lifecycle 16
Rate limits 16
Time: demo clock and live clock 17
Demo days and live businesses 17
Migrations, in order 17
Function catalogue 18
The frontend 19
Routes and who sees them 19
Data flow in the app 19
Accounts and sign-in 19
Design system 20
PWA, offline and headers 21
Performance and accessibility 21
The Lab and the splash 21
Testing and verification 23
Deployment and operations 24

Run it locally 24
Release rules the team follows 24
Observability 24
What works today 25
Known limits and the next engineering steps 25
Reference 27
Codes you will see 27
Technical questions judges ask 27

01 · SUMMARY

Refill on one page

Refill is an appointment recovery desk. When a client cancels, it offers the empty slot to the waiting clients who fit it and said yes to messages. The first to accept gets it, and only one booking can ever land in a slot. Every decision is logged with a reason.

The demo business is a salon (Kesar Salon, Jubilee Hills), but the wording and the data model are business-neutral: staff, services, clients. A tutoring centre and a spa already run on it as test businesses.

The three case checks and where each is proven

Check (points)	How Refill guarantees it	Proof
A 60-minute request cannot occupy a 30-minute slot (10)	`reasonFor()` returns `DURATION_TOO_LONG`; the database function `fit_reason()` re-checks inside `send_offers()` and again inside `accept_offer()`.	Tests C01, C02 · Lab check 1
Two requests for one slot: one confirmed booking, one waitlist entry (10)	`accept_offer()` locks the offer, the slot and the client row in a fixed order (`FOR UPDATE`). A partial unique index allows one confirmed booking per slot.	Tests C04, C09, C14, C15 · `npm run race` · Lab live race
A client without contact consent is excluded from outreach (5)	Consent is checked first; `null` (never asked) counts as no. Re-checked when offers are sent and again at the moment of booking. STOP revokes every open offer.	Tests C05, C06, C12 · Lab check 3

Refill in numbers

What	Count
Database functions (PL/pgSQL and SQL)	83
Tables (row level security on, no policies)	13
Migrations, applied in order	11 (001 to 011), about 3,000 lines of SQL
Frontend source (TypeScript, TSX, CSS)	about 8,800 lines
Unit and scenario tests (`npm test`)	51 passing
Live database checks (`npm run dbtest`)	28
Cross-role flow checks (`npm run flowtest`)	14
Race test (`npm run race`)	8 simultaneous taps, 3 runs, exactly 1 booking each run
Main JavaScript bundle	274 kB raw, 86 kB gzipped; pages load lazily

02 · THE SHAPE OF THE SYSTEM

Architecture

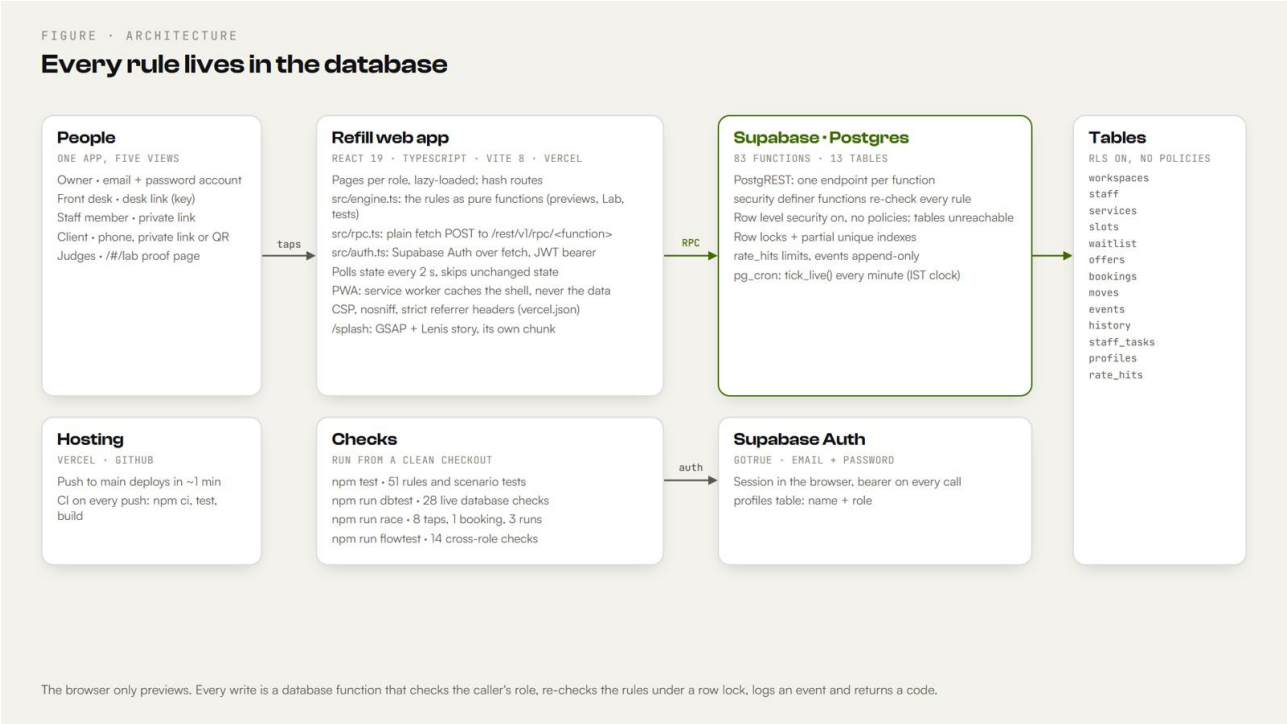


Figure 1. The browser previews; the database decides. Every write is a function that checks the role, re-checks the rules under a row lock, logs an event and returns a code.

Thin client, thick database

Refill has no application server of its own. The web app is a static single-page app on Vercel. It talks straight to Supabase, which gives us Postgres, an HTTP layer over it (PostgREST), authentication (GoTrue) and a scheduler (pg_cron).

The important decision: **the browser is never trusted.** It can preview anything (the Lab and the desk both run the rules engine locally to show who fits), but nothing it decides is written as-is. Every write is a Postgres function that:

- checks who is calling (owner account, front-desk key, staff key or client token);
- checks a rate limit for that caller;
- takes row locks in a fixed order;
- re-checks every rule against the current rows (consent, skill, duration, time, hold window, message cap);
- writes the change and an append-only event;
- returns a JSON result with an outcome code the page can show in plain words.

> **Why this matters**
> Two phones can both read "free" at the same instant. If the app checked availability and then booked, both would book (time-of-check to time-of-use). Moving the check and the write into one locked database transaction removes that window entirely.

One request, step by step

1. A page calls `rpc(fn, args)` in `src/rpc.ts`.
2. That is a plain `fetch` POST to `https://<project>.supabase.co/rest/v1/rpc/<fn>` with the publishable key as `apikey`, and, when someone is signed in, `Authorization: Bearer <access token>`.
3. PostgREST maps the call onto the Postgres function of the same name. Only functions granted to `anon` / `authenticated` are reachable.
4. The function runs `security definer` with a pinned `search_path` (`public, pg_temp`), so it can touch tables the caller cannot, and cannot be tricked by a malicious schema on the path.
5. The result comes back as JSON. Errors raised in SQL (`RATE_LIMITED`, `TOO_LARGE`, role failures) become an `ApiError` with a human message.

Where state lives

State	Where	Notes
Bookings, offers, waitlist, staff, events	Postgres	The single source of truth. Never cached by the service worker.
Signed-in session	`localStorage` key `refill.session`	Access + refresh token from Supabase Auth; refreshed before expiry.
Demo or account mode, remembered name and email	`localStorage` (`refill.mode`, `refill.name`, `refill.email`)	Only prefills; the app works without them (private mode).
Owner / desk / staff / client keys	In the private link (URL)	Only a SHA-256 hash of each key is stored in the database.
UI state (current page, open sheet)	React state + the URL hash	Hash routes, so every view is linkable.

03 · WHAT IT IS BUILT WITH, AND WHY

The stack

Layer	Choice	Version	Why
UI	React + TypeScript (strict)	React 19.2 · TS 6.0	Components per role; types shared with the engine and the scripts.
Build	Vite + @vitejs/plugin-react	Vite 8.3	Instant dev server; code splitting per page; CSV files imported as text.
Tests	Vitest	5.0	Engine unit tests and JSON scenario files.
Lint	oxlint	1.81	Fast lint in CI and locally.
Database	Supabase Postgres	Free plan	Transactions, row locks, partial unique indexes, PL/pgSQL.
API	PostgREST (via Supabase)	-	Every SQL function becomes an HTTP endpoint; no server code to host.
Auth	Supabase Auth (GoTrue) over REST	-	Email + password; email confirmation off for the demo project.
Scheduler	pg_cron	-	`tick_live()` every minute moves the live clock (India time).
Hosting	Vercel	-	Push to `main` deploys in about a minute; rewrites, cache and security headers in `vercel.json`.
CI	GitHub Actions	Node 22	`npm ci`, `npm test`, `npm run build` on every push and pull request.
QR codes	qrcode	1.5	Offer and sign-up QR codes on the desk.
Story page	GSAP + ScrollTrigger, Lenis	3.15 · 1.3	Only on `/splash`, in its own chunk; the app never loads them.
Fonts	Satoshi, Clash Display, JetBrains Mono	-	Fontshare + Google Fonts, allowed by the CSP.

> **What we deliberately did not add**
> No Supabase JS client (plain `fetch` is enough and keeps the bundle small), no component kit, no CSS framework, no global state library, no animation library in the app itself. Every dependency is listed above.

04 · WHERE EVERYTHING IS

Repository map

Path	What it holds
`src/engine.ts`	The rules engine: validation, fit test, matching, race, notice, plain-language reasons. Pure functions, no I/O.
`src/engine.test.ts`	Engine tests, the organiser fixture (W1 → S2), and every scenario file.
`src/api.ts`	Types for every database row, the `useBook` state hook (polling), role keys, conversions to engine types.
`src/rpc.ts`	The one function that calls the database (`rpc`), and `ApiError` with plain messages.
`src/auth.ts`	Sign up, sign in, refresh and sign out against Supabase Auth over REST.
`src/config.ts`	Supabase URL and publishable key (overridable with `VITE_SUPABASE_URL` / `VITE_SUPABASE_KEY`).
`src/day.ts`	Builds the demo salon day from the CSV files in `data/salon/`.
`src/App.tsx`	Routes, the shell (top bar, role pill, dock), the Home page.
`src/desk/`	Front desk: `Today.tsx`, `SlotSheet.tsx` (every action on one appointment), `Waitlist.tsx`, `Activity.tsx` (evidence log), `common.tsx`.
`src/Owner.tsx`	Owner: Overview, Team, Rules, History.
`src/StaffView.tsx`	A staff member's own day and "I can't come in today".
`src/Join.tsx`, `Me.tsx`, `Offer.tsx`, `Move.tsx`, `MovePicker.tsx`	Client pages: sign-up with consent, own page, offer, choose a new time.
`src/ClientHome.tsx`, `Directory.tsx`, `Manage.tsx`	Client home (places), the directory of businesses, managing a booking.
`src/SignIn.tsx`, `Setup.tsx`, `Settings.tsx`	Accounts: sign-in/sign-up, business setup, account and business settings.
`src/Lab.tsx`, `src/liveRace.ts`	Proof page for judges: case checks live, fixture, scenarios, edge toggles, live database race.
`src/Dial.tsx`	The Day Dial (the day as a watch face) and its sub-dials.
`src/styles.css`, `src/motion.css`, `src/motion.tsx`, `src/ui.tsx`	Design tokens, motion layer, icons, shared UI pieces.
`src/story/`	The `/splash` story page (GSAP + Lenis), its own dial and facts.
`data/salon/*.csv`	Synthetic demo data: 4 staff, 9 services, 26 appointments, 17 on the waitlist.
`data/fixture/C29.fixture.json`	The organiser fixture, kept byte for byte.
`supabase/migrations/001…011`	The whole database: tables, indexes, functions, grants. Applied in order.
`scripts/race-test.ts`, `db-test.ts`, `flow-test.ts`	Tests against the live database with the public key.
`tests/core-scenarios.json`, `tests/scenarios.json`	23 core scenarios and 12 more from Parth, run by `npm test`.
`public/sw.js`, `manifest.webmanifest`	Installable PWA: service worker and app manifest.
`vercel.json`	Rewrites for `/splash`, cache headers, security headers (CSP).

Path	What it holds
`.github/workflows/ci.yml`	CI: install, test, build.
`docs/`	Demo script, known limits, setup check, market shocks, business explainer, team tasks.

05 · SRC/ENGINE.TS

The rules engine and its algorithms

The engine is a set of pure functions with no I/O. The desk preview, the Lab, every test and the scripts all call it. The database implements the same rules again in SQL (`fit_reason()`), so the engine is for explaining and previewing, and the database is for deciding.

Data model

Type	Fields	Meaning
`Slot`	id, start ("HH:MM"), minutes, skill, status (free · booked · blocked), staff?, customer?, service?, price_inr?	A time on someone's day. `free` slots are the gaps to fill; `blocked` slots are held (for example, a staff member is out).
`Entry`	id, minutes, skill, consent (true · false · null), name?, service?, price_inr?, free_from?, free_until?	A client on the waitlist: what they want, how long it takes, whether they said yes to messages, when they are free.
`Staff`	id, name?, skills[], present, shift_start?, shift_end?	Who works a slot (v2). A freed slot is that person's time, so it fits any service they can do.
`Input`	slots[], waitlist[], staff[]?	One day.

Validation: a bad file never gives a half-right answer

`validate(raw)` checks untrusted JSON (a pasted file, an import, a scenario) and returns typed input or the first problem. It accepts the organiser fixture shape (`{ records: { slots, waitlist } }`) too.

- Required fields: slots need id, start, minutes, skill, status; entries need id, minutes, skill, consent; staff need id, skills, present.
- Times must match `^([01]\d|2[0-3]):([0-5]\d)$`; minutes must be positive whole numbers; status must be free, booked or blocked; consent must be true, false or null.
- Duplicate ids in any list stop the run with `DUPLICATE_ID`; a slot naming an unknown staff member stops it with `INVALID_VALUE`.
- Error codes: `MISSING_FIELD`, `INVALID_VALUE`, `DUPLICATE_ID`, each with a JSON path (`slots[3].start`) and a sentence.

The fit test: one reason, in a fixed order

`reasonFor(entry, slot, staff)` returns the **first** rule a client breaks for a slot, or `null` if they fit. The order is fixed so every skip has exactly one reason and the evidence log is deterministic:

```
export function reasonFor(entry, slot, staff) {
  if (entry.consent !== true) return 'NO_CONSENT'          // null (never asked) counts as no
  if (staff ? !staff.skills.includes(entry.skill)          // v2: what the staff member can do
            : entry.skill !== slot.skill) return 'SKILL_MISMATCH'
  if (entry.minutes > slot.minutes) return 'DURATION_TOO_LONG'  // 60 min never fits 30
  const start = toMin(slot.start)
  if (entry.free_from && start < toMin(entry.free_from)) return 'NOT_AVAILABLE'
  if (entry.free_until && start + slot.minutes > toMin(entry.free_until)) return 'NOT_AVAILABLE'
  return null
}
```

Consent is first on purpose: a client who never said yes is never even evaluated for anything else, so their data is not used for outreach decisions.

A shorter service may take a longer slot (a 30-minute haircut fits a 60-minute gap). A longer one never takes a shorter slot.

Matching: greedy, in queue order, nobody twice

`matchAll(input, opts)` fills every free slot from the waitlist:

1. Walk the slots in listed order; skip anything not `free`.
2. If the slot starts sooner than `now + minLead` (10 minutes by default), mark it `tooSoon`. Nobody can get there in time.
3. If the staff member who works it is absent, mark it `staffAbsent`.
4. Otherwise split the waitlist into `eligible` (in queue order) and `excluded` (each with its reason) using `reasonFor()`.
5. Choose the first eligible client who has not already been matched; record the pair.

Cost: one pass over the waitlist per free slot, so **O(S × W)** for S free slots and W clients. A busy salon day is tens of slots and tens of clients, so this is instant.

> **Why greedy and not "optimal" matching**
> A maximum bipartite matching (or the Hungarian algorithm) could fill more slots in rare cases by giving someone a later place than their queue position. We chose **fairness and explainability** instead: whoever has waited longest and fits is first, and the reason for every decision fits in one sentence. This is a product decision, and the desk can still offer any slot to anyone who fits.

In the live app the desk works one gap at a time: it sees the eligible list and the skipped list for that slot, then sends offers to the first few (3 by default). The database decides who books.

Races: the same rule as the database

`race(input, slotId, requests)` simulates several clients accepting one slot in a given order. The first eligible request is confirmed; later eligible ones keep their waitlist place; ineligible ones are rejected with their reason; an unknown id is `UNKNOWN_ID`; a repeated id is the same request (it is de-duplicated). This mirrors what `accept_offer()` does with a row lock, so the Lab can show the expected answer next to the real one.

Notice: normal, late or past

```
export function notice(slotStart, now, cutoff = 120): 'NORMAL' | 'LATE' | 'PAST' {
  const lead = slotStart - now
  if (lead < 0) return 'PAST'
  return lead < cutoff ? 'LATE' : 'NORMAL'      // under 2 hours is a late cancellation
}
```

Words for people

`explain(reason, entry, slot, staff)` turns a code into one sentence for the evidence view ("Needs 90 min; the gap is 60 min"). `LABEL` gives the short words a business owner reads first ("Needs more time", "No permission to message"); the code stays alongside for the record.

Worked example: Kabir's 12:00 opens up

At 11:15 Harsh cancels his 12:00 hair spa (60 minutes) with Kabir, who does hair and nails. Seventeen clients are waiting. The engine gives 6 who fit and 11 who are skipped, each with one reason:

Client	Wants	Result	Why
Meera	Hair spa · 60	Fits	-
Sameer	Manicure · 45	Fits	-
Omar	Haircut · 30	Fits	-
Tanvi	Gel polish · 60	Fits	-
Vikram	Blow-dry · 30	Fits	Free 10:00–13:00, so 12:00–13:00 works
Ira	Manicure · 45	Fits	-
Arnav, Fatima	Hair spa, Hydra facial	`NO_CONSENT`	Never asked about messages (null)
Nisha, Yusuf	Blow-dry, Root colour	`NO_CONSENT`	Said no to messages
Rhea, Naina	Facials (skin)	`SKILL_MISMATCH`	Kabir does hair and nails
Leela, Aarav	Root colour 90, Global colour 120	`DURATION_TOO_LONG`	The gap is 60 minutes
Zoya, Kiran, Priya	Haircut, Hair spa, Haircut	`NOT_AVAILABLE`	Free only 13:00–17:00, 17:00–20:00, 09:00–12:00

The desk sends 3 offers (Meera, Sameer, Omar). Meera and Sameer tap Book at the same instant; Meera books, Sameer keeps his place. ₹1,600 comes back into the book.

Scenario files and the organiser fixture

`runScenario(s)` runs one JSON scenario through the engine and lists every disagreement with its `expect` block (matches, exclusions with codes, input errors, race outcomes). `tests/core-scenarios.json` has 23 scenarios, `tests/scenarios.json` has 12 more. The organiser fixture `data/fixture/C29.fixture.json` reproduces its own `expected_match` (W1 → S2) in `src/engine.test.ts`.

06 · SUPABASE/MIGRATIONS

The database

The database is the product's core. It holds every guarantee: who may do what, the rules, one booking per slot, consent, holds, caps, rate limits and the audit log. Eleven migrations build it, each additive, so the deployed client keeps working between a migration and the next deploy.

Security model

- **Row level security is on for every table, with no policies.** The public (publishable) key therefore cannot read or write any table directly. Table privileges are also revoked from `anon` and `authenticated`.
- **Every access goes through a function.** Functions are `security definer` with `set search_path = public, pg_temp`. Only the functions a page needs are granted to `anon` / `authenticated`; internal helpers are not callable.
- **Roles are proven on every call.** Owner (a signed-in account that owns the business, or the owner key for demo days), front desk (a desk key), staff member (their own key), client (a private token). Helpers: `role_of()`, `require_owner()`, `require_desk()`, `require_staff()`, `is_owner()`.
- **Keys are never stored.** `new_key()` makes a random key; only `hash_key()` (SHA-256, hex) is kept. A leaked database row does not leak a working link.
- **Accounts** (migration 009): Supabase Auth users; `profiles` holds name and role (owner or client); `workspaces.owner_user` ties a live business to its owner; `waitlist.user_id` ties a client's entries to their account.
- **The audit log cannot be edited.** A trigger on `events` raises `EVENTS_ARE_APPEND_ONLY` on any update or delete.

Tables

Table	Key columns	Purpose
`workspaces`	id, salon, kind (demo · live), now_min, day, late_cutoff_min, min_lead_min, hold_min, max_offers, owner_hash, desk_hash, owner_user, listed	One business or one demo day, with its rules and clock.
`staff`	ws, id, name, skills[], present, shift_start, shift_end, key_hash, active	Team members, their skills, working hours and in/out.
`services`	ws, id, name, minutes, skill, price_inr	What a live business offers.
`slots`	ws, id, start_min, minutes, skill, status, staff_id, customer, service, price_inr, refilled, cancelled_customer, blocked_reason	The day's appointments and gaps.
`waitlist`	ws, id, pos, name, minutes, skill, consent, consent_at, consent_source, free_from, free_until, status, user_id	Clients waiting, with consent and its source and time.
`offers`	id (uuid), ws, slot_id, entry_id, status, expires_min, created_at, responded_at	Every offer and how it ended.
`bookings`	id, ws, slot_id, entry_id, offer_id, status	Confirmed bookings made from offers.

Table	Key columns	Purpose
`moves`	id, ws, slot_id, customer, reason, status, to_slot	Appointments that need a new time (staff out, client or business request).
`events`	id, ws, at, kind, code, slot_id, entry_id, booking_id, detail (jsonb)	The append-only evidence log.
`history`	ws, day, start_min, minutes, service, staff_name, customer, status, price_inr, refilled	Past days of live businesses (archived every night).
`staff_tasks`	ws, staff_id, text, done	Small to-dos for staff.
`profiles`	user_id, name, role	Account name and role.
`rate_hits`	key, window_start, hits	Fixed-window rate limit counters.

Indexes and constraints that carry guarantees

Index / constraint	What it guarantees
`bookings_one_per_slot`: unique (ws, slot_id) where status = 'confirmed'	Never two confirmed bookings in one slot, even if every other check failed.
`offers_one_open`: unique (ws, slot_id, entry_id) where status = 'sent'	One open offer per client per slot; re-sending is idempotent.
`moves_one_open`: unique (ws, slot_id) where status = 'open'	One open move per appointment.
Check constraints on workspace settings	now_min 0–1439, late_cutoff_min 0–1440, min_lead_min 0–240, hold_min 1–240, max_offers 1–20.
`events` append-only trigger	The log cannot be rewritten.
Indexes on (ws), (ws, id desc), (owner_user), (user_id), history (ws, day desc)	Fast per-business reads.

Concurrency: how one booking per slot is guaranteed

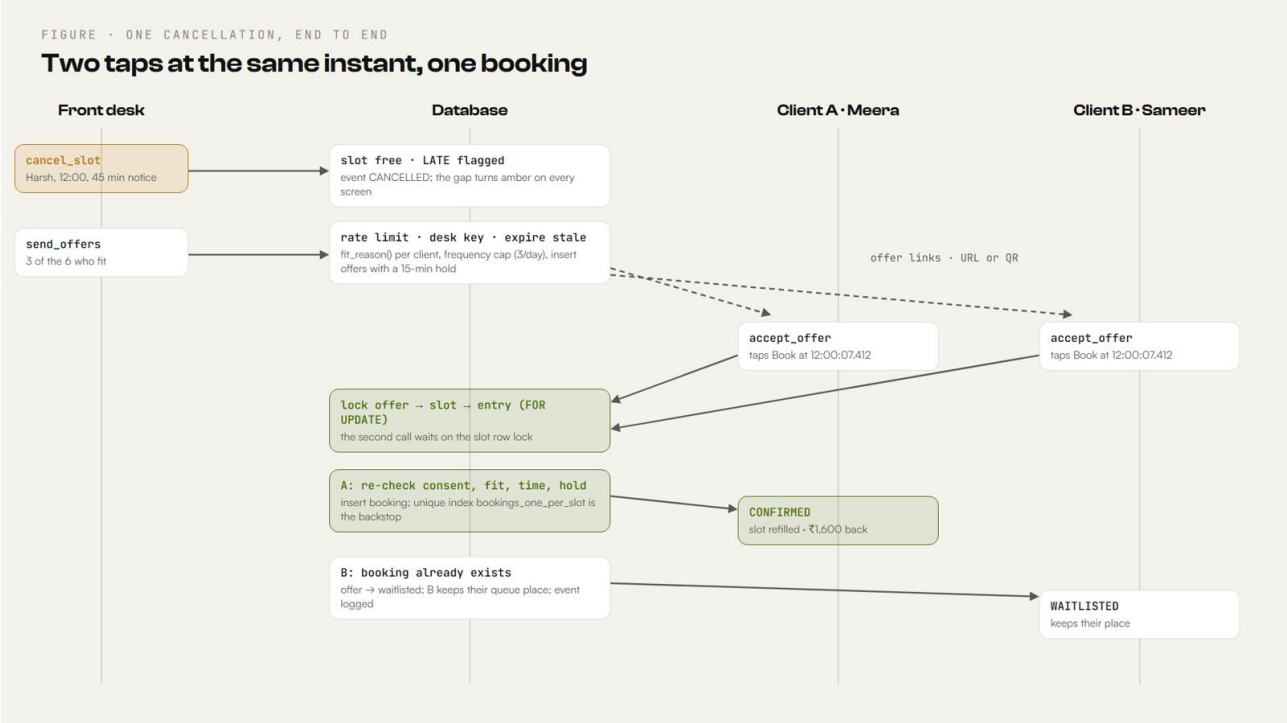


Figure 2. One cancellation, end to end. Two taps at the same millisecond queue on the slot row lock; the first books, the second keeps its place.

Every function that changes several rows locks them in **one global order: moves, staff, offers, slots, waitlist.** Two transactions can then never wait on each other in a circle, so there are no deadlocks.

`accept_offer(p_offer)` in detail:

1. Rate limits: 600 taps a minute per IP, 30 a minute per offer.
2. `select … for update` on the offer, then the slot, then the client's waitlist row. A second caller for the same slot now waits.
3. If the offer is already accepted, return `ALREADY_CONFIRMED` and the same booking (a double tap books once).
4. If it was declined, revoked, waitlisted or expired, return that state.
5. If the hold window has passed, mark it expired (`OFFER_EXPIRED`).
6. Re-check consent (`NO_CONSENT`), that the client is still waiting (`ALREADY_BOOKED`), `fit_reason()`, and that the slot is still open and in the future (`SLOT_UNAVAILABLE`). A failure revokes the offer and logs why.
7. Insert the booking. The partial unique index is the backstop: a `unique_violation` is caught and treated as "someone else got there first".
8. If a booking already exists, mark this offer `waitlisted`; the client keeps their queue place and is told so.
9. Log every outcome in `events`.

Offers: hold window, cap and lifecycle

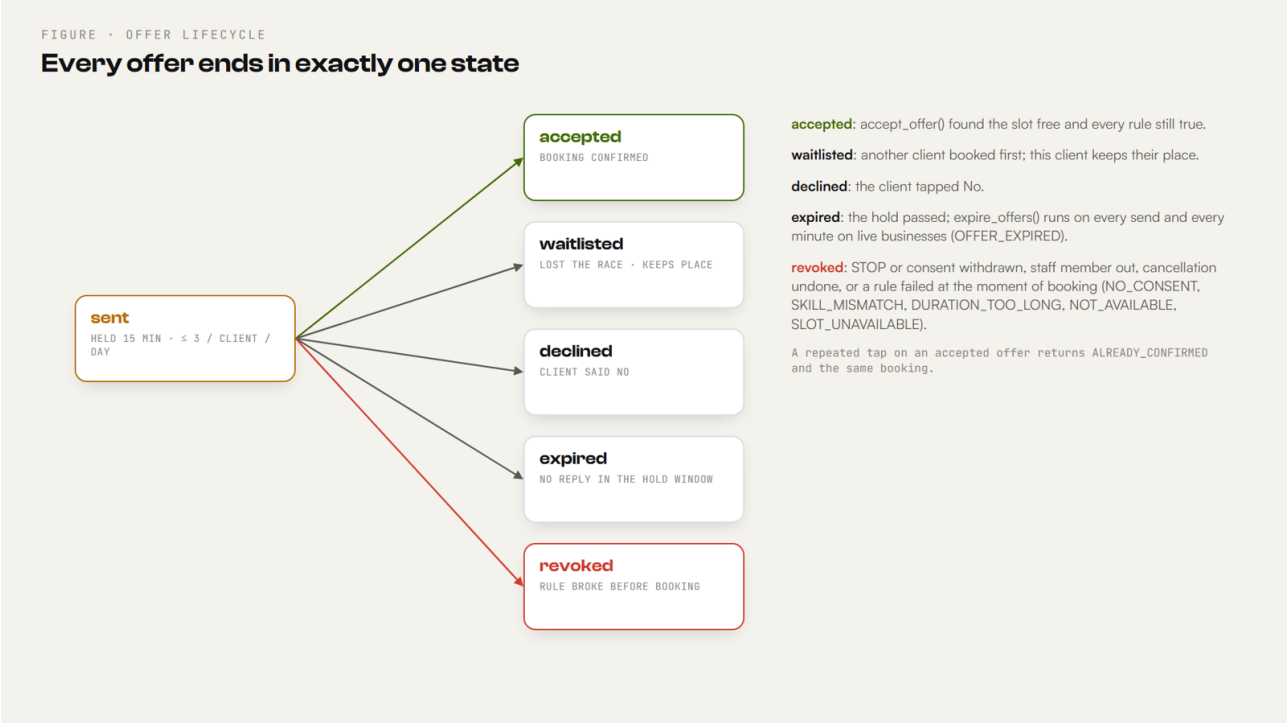


Figure 3. Every offer ends in exactly one state.

- **Hold window.** `expires_min = least(now + hold_min, slot_start - min_lead_min)`: 15 minutes by default, but never past the point where the client could still arrive.
- **Expiry.** `expire_offers(ws, now)` marks overdue offers `expired` with `OFFER_EXPIRED / NO_REPLY` events. It runs on every `send_offers()` and every minute for live businesses.
- **Message cap.** Before sending, `send_offers()` counts offers already sent to that client; at `max_offers` (3 by default) it skips them with `FREQUENCY_CAP`. Nobody is nagged.
- **Lead time.** A slot starting within `min_lead_min` (10) is refused with `SLOT_TOO_SOON`.
- **Payload cap.** More than 50 clients in one call raises `TOO_LARGE`.

Rate limits

`enforce_rate(key, limit, seconds)` keeps a counter per key per fixed window in `rate_hits` (insert or increment) and raises `RATE_LIMITED` above the limit. Keys combine an action with a caller (the IP from the request headers via `client_ip()`, the workspace, or the offer).

Key	Limit	Protects
`create:<ip>` / `create:all`	120 per 10 min per IP · 1,500 per 10 min overall	Making demo days
`reset:<ip>`	30 per 10 min	Resetting a demo day
`ws:<workspace>`	240 per min	Desk and owner actions on one business
`tap:<ip>` · `offer:<id>`	600 per min · 30 per min	Accepting offers
`join:` · `me:` · `move:` · `book:`	30 / 10 min · 120 / min · 30 / min · 30 / 10 min	Client sign-up, client pages, moves, self-booking

Key	Limit	Protects
`profile:` · `dir:` · `claim:`	60 / 10 min · 240 / min · 30 / 10 min	Accounts, directory, claiming entries

Retention: demo workspaces are deleted after 3 days and old rate counters after an hour (`cleanup()`). A global cap stops the project at 20,000 workspaces.

Time: demo clock and live clock

- **Demo days** keep their own clock (`now_min`, starting at 11:15 in the story). `set_clock()` moves it, expiring offers as it passes them, so a judge can replay a day.
- **Live businesses** run on India time. `pg_cron` calls `tick_live()` every minute: it archives yesterday into `history` on the first tick of a new day, sets `now_min` from `ist_minute()`, and expires overdue offers.

Demo days and live businesses

Migration 007 split workspaces into **demo** (made from the sample salon, synthetic, deleted after three days, demo clock) and **live** (made empty by its owner, any kind of service business, kept, real clock). Live businesses have their own services, staff, working hours, tasks and history. Migration 010 adds a directory (listed by default, owner can switch it off) that shows only business-level facts, never a client's name. Migration 011 lets clients book an open time themselves (`open_times`, `book_time`), with the waitlist as the fallback, and lets owners rename or delete a business and anyone delete their own account.

Migrations, in order

#	What it added
001	Core tables, `fit_reason`, `send_offers`, `accept_offer` with row locks, the two partial unique indexes, RLS on with no policies.
002	Rate limits, retention, stricter input, the offer hold window, the per-client message cap, undo for a mistaken cancellation, append-only events.
003	Offers that lapse as the clock passes them; tighter privileges.
004	Roles (owner, staff) proven by hashed keys; staff-aware matching; staff absence; client self sign-up with consent.
005	Working hours; moving appointments (desk picks a time, or the client gets a link); a front-desk role; owner policies.
006	The client's offer page and sign-up tell the truth about cancelled and withdrawn offers.
007	Live businesses next to demo days: services, tasks, history, India time.
008	Client pages know whether they belong to a demo or a live business.
009	Accounts: Supabase Auth users, profiles, owner accounts, client entries tied to accounts.
010	Directory of businesses; account actions on a client's own entries.
011	Clients book open times themselves; the desk books a waiting client; the live clock on pg_cron; delete business and account.

Function catalogue

All 83 functions, grouped by what they are for (internal helpers are not callable from the browser):

Area	Functions
Rules and helpers	`fit_reason`, `log_event`, `apply_consent`, `client_ip`, `enforce_rate`, `cleanup`, `events_append_only`, `hash_key`, `new_key`, `next_id`, `ist_minute`, `ist_date`
Roles	`role_of`, `is_owner`, `require_owner`, `require_desk`, `require_staff`
Workspaces and setup	`create_workspace`, `reset_workspace`, `load_day`, `create_business`, `update_business`, `delete_business`, `set_policy`, `set_listed`, `get_state`, `set_clock`, `sync_clock`, `tick_live`, `archive_day`, `get_history`
Team and services	`save_staff`, `remove_staff`, `save_service`, `remove_service`, `staff_link`, `desk_link`, `get_staff_day`, `set_staff_presence`, `add_task`, `set_task`, `remove_task`
The day	`cancel_slot`, `do_cancel`, `restore_slot`, `reassign_slot`, `add_appointment`, `open_times`, `book_time`
Offers	`send_offers`, `accept_offer`, `decline_offer`, `expire_offers`, `get_offer`
Moves	`move_options`, `apply_move`, `open_move`, `get_move_options`, `move_booking`, `move_link`, `get_move`, `choose_move`, `cancel_my_booking`, `my_move`
Waitlist and consent	`add_entry`, `set_consent`, `stop_messages`, `join_info`, `join_waitlist`, `get_my_entry`, `set_my_consent`, `leave_waitlist`
Accounts and directory	`get_profile`, `set_profile`, `my_businesses`, `my_entries`, `my_entry`, `my_entry_move`, `my_entry_leave`, `my_entry_consent`, `claim_entries`, `directory`, `delete_account`

07 · SRC/

The frontend

Routes and who sees them

Route	Page	Who
`/#/`	Sign in / sign up, or Home for the signed-in role	Everyone
`/#/desk`, `/#/desk/slot/<id>`	Today: needs-you-now queue, team in/out, the day as a dial or calendar; one sheet with every action on an appointment	Front desk
`/#/desk/waitlist`, `/#/desk/activity`	Waitlist with consent per person; evidence log with filters and CSV/JSON export	Front desk
`/#/owner`, `/#/owner/team`, `/#/owner/rules`, `/#/owner/history`	Money recovered and at risk, staff load; team and links; rules; past days	Owner
`/#/s/<ws>/<staff>/<key>` · `/#/staff/<id>`	A staff member's own day (private link) · the owner's preview of it	Staff
`/#/join/<code>`	Client sign-up with an explicit yes or no to messages (never pre-ticked)	Client
`/#/me/<token>` · `/#/me`	A client's own page · their places	Client
`/#/o/<uuid>` · `/#/m/<uuid>`	An offer (Book or No) · choose a new time	Client
`/#/places`	Directory of listed businesses	Anyone
`/#/lab`	Proof: case checks live, fixture, scenarios, edge toggles, live race	Judges
`/#/settings`	Account and business settings	Signed in
`/splash`	The scroll story (a real path, rewritten to `index.html` by Vercel)	Anyone

Data flow in the app

- `useBook` (in `src/api.ts`) loads the business with `get_state` and polls it **every 2 seconds**, skipping a re-render when nothing changed. Live businesses also sync the clock every minute.
- Every action calls one database function and then refreshes. The page never edits its copy of the data and hopes; it shows what the database says.
- The desk preview runs the engine locally (`asSlot`, `asEntry`, `asStaff` convert database rows), so "who fits" appears instantly; the database re-checks when offers are actually sent.
- Errors come back as codes and are shown in plain words (`LABEL` in the engine, `SAID` in `auth.ts`, messages in `rpc.ts`).

Accounts and sign-in

1. `SignIn.tsx`: sign up (name, email, password) or sign in. `auth.ts` calls Supabase Auth `/auth/v1/signup` or `/auth/v1/token?grant_type=password` with `fetch` and saves the session.

2. An `AUTH_EVENT` makes the app re-route itself: a new account picks a role (`Welcome`: "I run a business" or "I book appointments", stored with `set_profile`).
3. An owner sets up the business (`Setup.tsx` then `create_business`); a client goes to their places.
4. Every later database call carries the access token; it is refreshed before it expires. Sign-out clears it.
5. The sign-in page (v8) adds: a switch link instead of tabs, show-password, a Caps Lock note, a strength sub-dial on sign-up, inline errors tied to fields with `aria-describedby`, the remembered email, and autofocus only when a mouse is present (so a phone keyboard does not cover the page).

Design system

The look is a **chronograph**: the day as a watch face, dark first, with a hand-tuned light theme. Colour always carries meaning.

Token	Dark	Light	Meaning
canvas · surface	#0a0a0c · #131316	#f4f2ed · #ffffff	Page and raised surfaces
ink · ink-2 · ink-3	#f4f1ea · #a9a59c · #6f6c66	#121214 · #5a5852 · #8c8981	Text, secondary text, hints
act (lime)	#c6f24e	#3f6b00	You can act; also "refilled"
beat (amber)	#ffb547	#d4820a	An open gap (it pulses)
danger (coral)	#ff5d4d	#d13a2b	Someone needs moving, or a staff member is out

- Type: Clash Display (headlines, big numbers), Satoshi (UI and text), JetBrains Mono (times, codes, labels).
- Shape: concentric radii 20 / 12 / 8; glass only on what floats (bar, dock, sheets); film grain at about 4%.
- **The Day Dial** (`src/Dial.tsx`): 10:00 to 20:00 clockwise over 300 degrees, one ring per staff member, one arc per appointment. Angle = −150° + (minute − 10:00) / 600 × 300°. Amber gaps pulse, lime refills draw in, coral needs moving, a hand marks the clock. Every arc opens its appointment.
- Motion (`motion.tsx`, `motion.css`): view transitions, ambient glow that follows the day, staggered rise, spring presses, count-up numbers. All of it switches off under reduced motion.

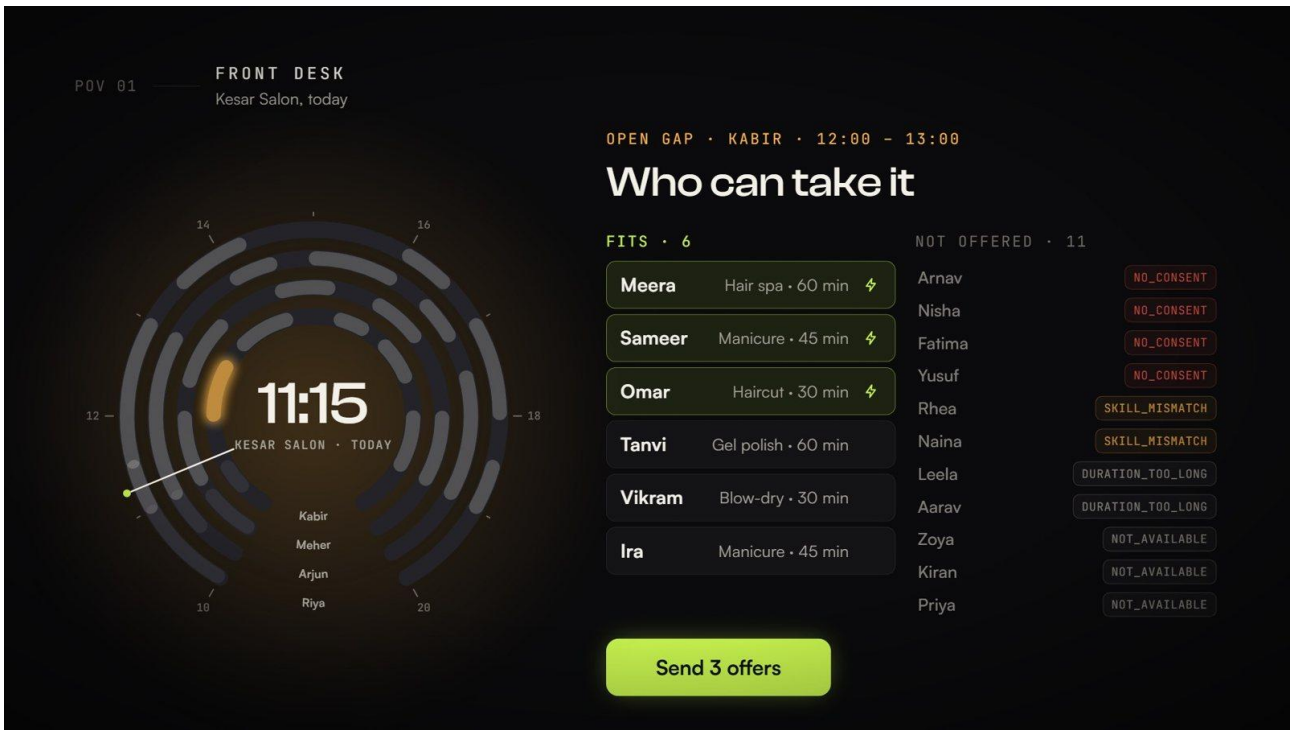


Figure 4. The front desk: the Day Dial on the left, who fits and who is skipped (with codes) on the right.

PWA, offline and headers

- `public/sw.js` caches the app shell (`refill-shell-v3`). Navigation is network-first, so a new deploy shows at once; the cached shell is used only offline. **Requests to Supabase are never cached**: bookings are always live.
- Installable from the browser menu ("Install app" / "Add to Home Screen").
- `vercel.json` sets a Content-Security-Policy (scripts only from self; fonts from Fontshare and Google; connections to our Supabase project), `X-Content-Type-Options: nosniff`, a strict referrer policy, and a Permissions-Policy that turns off camera, microphone and geolocation. Hashed assets are cached for a year; `sw.js` is never cached.

Performance and accessibility

- Pages are lazy-loaded; the main bundle is 86 kB gzipped. The splash page's GSAP and Lenis live in their own 63 kB (gzipped) chunk that only `/splash` loads.
- Inputs are 16 px (no zoom on iPhone), touch targets at least 42 px, no horizontal scroll at 375 px; checked in light and dark themes.
- Labels on every field, `aria-invalid` and `role="alert"` on errors, `aria-live` updates, reduced-motion support.

The Lab and the splash

`/#/lab` is for judges. It runs the three case checks live, loads the organiser fixture, runs every scenario file, lets you edit any input, flips edge cases (late cancellation, staff absent, duration change, consent withdrawn) with a before/after export, and fires a real race at the real database (`liveRace.ts`).

`/splash` (built by Chirag, PR #22) tells one salon day on the Day Dial, scrubbed by the scroll with GSAP ScrollTrigger and Lenis: the gap, who fits, the race, staff out, and the proof figures, each naming the command that produces it.

08 · HOW WE KNOW IT WORKS

Testing and verification

Command	What it checks	Against
`npm test`	51 tests: engine units (validation, fit order, matching, race, notice), the organiser fixture (W1 → S2), 23 core scenarios, 12 extra scenarios, and that the demo CSVs still load.	The engine, offline
`npm run dbtest`	28 checks: what the public key can and cannot do (tables refused), payload and text limits, rate limiting, hold windows, the message cap, undo, owner, desk, staff and client roles, absence, shifts, moves.	The live database, public key
`npm run race`	8 clients tap Accept on one slot at the same moment, 3 runs. Pass = exactly one booking per run; everyone else keeps their place. `npm run race -- 20 5` for 20 clients, 5 runs.	The live database
`npm run flowtest`	14 checks: one day, every role. Each step acts as one role and reads what another role sees (does the desk's offer reach the client, does the client's booking reach the owner).	The live database
`npm run build`	Type-check (`tsc -b`) and production build.	The code
CI (GitHub Actions)	`npm ci`, `npm test`, `npm run build` on every push to main and every pull request.	Every change

> **Reproduce everything from a clean checkout**
> `npm ci` · `npm test` · `npm run dbtest` · `npm run race` · `npm run flowtest` · `npm run build`. Node 20 or newer. See `docs/SETUP_CHECK.md`.

Two kinds of proof matter here. **Offline tests** prove the rules are right. **Live tests** prove the database enforces them for anyone holding the public key, including someone trying to bypass the app.

09 · RUNNING IT

Deployment and operations

Run it locally

```
git clone https://github.com/sirgreaseball/refill && cd refill
npm ci            # installs exactly what package-lock.json says
npm run dev       # http://localhost:5173, talks to the shared demo database
npm test          # rules engine, fixture, scenarios
npm run build     # production build into dist/
```

To use your own Supabase project: apply `supabase/migrations/001` to `011` in order, then set `VITE_SUPABASE_URL` and `VITE_SUPABASE_KEY` (the publishable key). Enable `pg_cron` for the live clock.

Release rules the team follows

- **Push to `main` deploys** to fillthechair.vercel.app in about a minute. `main` is protected: pull request, review and the CI `test` check (the owner can push directly).
- **Database changes are additive.** Add functions and columns; `create or replace` with the same signature; never drop a signature the live client still calls. Every migration is saved as `supabase/migrations/NNN_name.sql` and the same SQL is applied.
- **Lock order** in every function: moves, staff, offers, slots, waitlist. Every write re-checks rules under lock.
- Teammates work on a branch and open a pull request; their AI assistants follow `AGENTS.md` (narrow file scopes, `npm ci` only, no direct database or deploy commands).

Observability

The `events` table is the operational log: every cancellation, offer sent or skipped (with its code), acceptance, rejection, expiry, move and undo, with the slot, client and booking ids. The desk's Activity page filters it and exports CSV or JSON (with a guard against spreadsheet formula injection).

10 · STATUS, HONESTLY

What works today

Feature	Status	Evidence
Fit rules: consent, skill (staff-aware), duration, availability	Live and tested	`npm test`, Lab checks 1 and 3, `dbtest`
One booking per slot under simultaneous taps	Live and tested	`npm run race`, Lab live race, tests C04/C09/C14/C15
Consent: yes / no / never asked; STOP revokes open offers	Live and tested	Tests C05, C06, C12; `dbtest`
Offer hold (15 min), expiry, message cap (3/day), lead time (10 min)	Live and tested	`dbtest`
Late cancellation flag (< 2 h) and undo of a mistaken cancellation	Live and tested	`dbtest`, Lab edge toggles
Staff absence: offers revoked, gaps held, clients flagged to move	Live and tested	`dbtest`, `flowtest`
Rescheduling: desk picks a time, or the client gets a link and picks	Live and tested	`dbtest`, `flowtest`
Roles: owner, front desk, staff, client, each proven by the database	Live and tested	`dbtest`, `flowtest`
Accounts (email + password), role choice, business setup	Live	Manual runs; new sign-in page v8
Live businesses: services, staff, hours, tasks, history, India-time clock	Live	Running test businesses (salon, tuition centre, spa)
Client self-booking of open times; directory of businesses	Live	Manual runs
Evidence log with CSV/JSON export	Live	Activity page
Installable PWA, light and dark themes, phone layouts	Live	Checked at 375 px and 1440 px
Messages to clients	Simulated	Offers are links and QR codes; no SMS or WhatsApp is sent
Booking future dates	Not yet	A live business works one day at a time
Forgot password, Google sign-in	Not yet	Need Supabase email (SMTP) and an OAuth app
Platform admin console, multi-location	Not yet	-
Market-shock rehearsal (`docs/SHOCKS.md`)	Written, not yet timed on live	Needs a person with a stopwatch

Known limits and the next engineering steps

- **Real messaging.** India requires DLT registration, approved sender IDs and templates for SMS; WhatsApp needs approved utility templates. The offer link is already the message body; a provider (SMS or WhatsApp) can be added behind `send_offers()` without changing the rules.
- **Future dates.** Extend `slots` with a date and the live clock with a calendar; the rules do not change.

- **Email features.** Configure Supabase SMTP for password reset and email confirmation.
- **Calendar sync.** Import from Google Calendar or a salon's existing booking tool so Refill sits on top of what a business already uses.
- **Admin and analytics.** A platform console and per-business dashboards over `events` and `history`.

11 · CODES AND ANSWERS

Reference

Codes you will see

Code	Where	Meaning
`NO_CONSENT`	fit, send, accept	The client has not said yes to messages (no, or never asked).
`SKILL_MISMATCH`	fit, send, accept	The staff member does not do that service.
`DURATION_TOO_LONG`	fit, send, accept	The service needs more time than the gap.
`NOT_AVAILABLE`	fit, send, accept	The client is not free then.
`FREQUENCY_CAP`	send	The client has had the maximum offers today.
`SLOT_TOO_SOON` · `SLOT_NOT_FREE` · `STAFF_ABSENT`	send	Too close to start · not a gap · the staff member is out.
`UNKNOWN_ENTRY` · `NOT_WAITING`	send	No such client · client no longer waiting.
`CONFIRMED` · `WAITLISTED` · `ALREADY_CONFIRMED`	accept	Booked · someone else booked first · a repeated tap.
`EXPIRED` · `REJECTED` · `NOT_FOUND`	accept	Hold passed · a rule failed (with its code) · no such offer.
`SLOT_UNAVAILABLE` · `ALREADY_BOOKED`	accept	Slot held or past · client booked elsewhere.
`OFFER_EXPIRED` / `NO_REPLY`	events	An offer lapsed without an answer.
`RATE_LIMITED` · `TOO_LARGE`	any call	Too many requests · too much in one call.
`DUPLICATE_ID` · `MISSING_FIELD` · `INVALID_VALUE`	engine validation	A bad input file.
`EVENTS_ARE_APPEND_ONLY`	database	Someone tried to edit the log.

Technical questions judges ask

Question	Answer
How do you guarantee two people never get the same slot?	Booking happens inside one database transaction that locks the offer, then the slot, then the client, in that order everywhere. Simultaneous taps queue on the slot lock; the first books, the rest find it taken and keep their place. A partial unique index allows one confirmed booking per slot as a backstop. `npm run race` and the Lab prove it live.
Why not check availability in the app first?	Two phones can both read "free" at the same instant (time-of-check to time-of-use). The app only previews; the database decides.
Could someone change data by calling the database directly?	No. Every table has row level security with no policies, and table privileges are revoked. Only checked functions are callable, each proves the caller's role, and rate limits stop floods. `npm run dbtest` checks exactly this with the public key.
What if a client taps Book twice?	The second call returns `ALREADY_CONFIRMED` and the same booking number.
How is consent handled?	Yes, no, or never asked; only yes may be messaged. Checked before any offer and again at booking. STOP, or switching messages off, revokes every open offer. Sign-up never pre-ticks consent.

Question	Answer
Is it only for salons?	No. The model is staff, skills, working hours, services, a waitlist and consent. A tuition centre and a spa already run on it.
What did you change after the business team's feedback?	Staff and duration constraints before outreach (staff-aware matching, working hours) and rescheduling instead of only cancelling. Building it exposed a real flaw: without working hours a client could be moved onto someone who had gone home; shifts now bound every move.